



מבוא למדעי המחשב מ'ח' (234114 \ 234117)

סמסטר חורף תש"פ

מבחן מסכם מועד א', 20 בפברואר 2020

2	3	4	1	1	
---	---	---	---	---	--

רשום/ה לקורס:

--	--	--	--	--	--	--	--	--	--

מספר סטודנט:

משך המבחן: 3 שעות.

חומר עזר: אין להשתמש בכל חומר עזר.

הנחיות כלליות:

- מלאו את הפרטים בראש דף זה ובדף השער המצורף, בעט בלבד.
- בדקו שיש 22 עמודים (4 שאלות) במבחן, כולל עמוד זה.
- כתבו את התשובות על טופס המבחן בלבד, במקומות המיועדים לכך. שימו לב שהמקום המיועד לתשובה אינו מעיד בהכרח על אורך התשובה הנכונה.
- העמודים הזוגיים בבחינה ריקים. ניתן להשתמש בהם כדפי טיוטה וכן לכתיבת תשובותיכם. סמנו טיוטות באופן ברור על מנת שהן לא תיבדקנה.
- יש לכתוב באופן ברור, נקי ומסודר, ובעט בלבד.
- בכל השאלות, הנכם רשאים להגדיר ולממש פונקציות עזר כרצונכם. לנוחיותכם, אין חשיבות לסדר מימוש הפונקציות בשאלה, ובפרט ניתן לממש פונקציה לאחר השימוש בה.
- אלא אם כן נאמר אחרת בשאלות, אין להשתמש בפונקציות ספריה או בפונקציות שמומשו בכיתה, למעט פונקציות קלט/פלט והקצאת זיכרון (`malloc`, `free`). ניתן להשתמש בטיפוס `bool` המוגדר ב-`stdbool.h`.
- אין להשתמש במשתנים סטטיים וגלובאליים אלא אם נדרשתם לכך מפורשות.
- כשאתם נדרשים לכתוב קוד באילוצי סיבוכיות זמן/מקום נתונים, אם לא תעמדו באילוצים אלה תוכלו לקבל בחזרה מקצת הנקודות אם תחשבו נכון ותציינו את הסיבוכיות שהצלחתם להשיג.
- נוהל "לא יודע": אם תכתבו בצורה ברורה "לא יודע/ת" על שאלה (או סעיף) שבה אתם נדרשים לקודד, תקבלו 20% מהניקוד. דבר זה מומלץ אם אתם יודעים שאתם לא יודעים את התשובה.
- נוסחאות שימושיות:

$$1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} = \Theta(\log n) \quad \log 1 + \log 2 + \dots + \log n = \Theta(n \log n)$$

$$1^k + 2^k + 3^k + \dots + n^k = \Theta(n^{k+1}) \quad 1 + k + k^2 + k^3 + \dots + k^n = \Theta(k^n), \quad k > 1$$

צוות הקורס 234114/7

מרצים: פרופ' תומר שלומי (מרצה אחראי), ד"ר יוסי ויינשטיין, ד"ר אהרון קופרשטוק, מר איהאב וותד
מתרגלים: עמית ברכה, דן ברקוביץ, גיא ברשצקי, עמית גנץ, עדי גרוס, עמר דהרי, קטרין חדאד, אברנם טל, תומר לנגה, גסוב מזאוי, עמר צברי, דמיטרי רבינוביץ' (מתרגל אחראי), אמאני שחאדה

בהצלחה!

[illegible]



שאלה 1 (25 נקודות):

א. (9 נקודות) חשבו את סיבוכיות הזמן והמקום של הפונקציה $f1$ המוגדרת בקטע הקוד הבא, כפונקציה של n . אין צורך לפרט שיקולים. חובה לפשט את הביטוי ככל שניתן. הניחו שסיבוכיות הזמן של $\text{malloc}(n)$ ו- free היא $\Theta(1)$, וסיבוכיות המקום של $\text{malloc}(n)$ היא $\Theta(n)$.

```
void f1(int n)
{
    int b = 2;
    while (n >= 1)
    {
        b *= b;
        n /= 2;
    }
    free(malloc(b));
}
```

סיבוכיות זמן: $\Theta(\quad)$ סיבוכיות מקום: $\Theta(\quad)$

ב. (8 נקודות) חשבו את סיבוכיות הזמן והמקום של הפונקציה $f2$ המוגדרת בקטע הקוד הבא, כפונקציה של n . אין צורך לפרט שיקולים. חובה לפשט את הביטוי ככל שניתן.

```
void f2(int n)
{
    for (int i = 1; i < n; ++i)
    {
        for (int j = 1; i * j < n; ++j)
            printf("*");
        printf("\n");
    }
}
```

סיבוכיות זמן: $\Theta(\quad)$ סיבוכיות מקום: $\Theta(\quad)$

[illegible]



ג. (8 נקודות): חשבו את סיבוכיות הזמן והמקום של הפונקציה f3 המוגדרת בקטע הקוד הבא, כפונקציה של n. אין צורך לפרט שיקולים. חובה לפשט את הביטוי ככל שניתן.

```
void aux(int n)
{
    if (n == 0) return;
    aux(n / 2);
    printf("%d", n % 2);
}

int f3(int n)
{
    for (int i = 1; i <= n; ++i)
    {
        aux(i);
        printf("\n");
    }
}
```

סיבוכיות זמן: $\Theta(\quad)$ סיבוכיות מקום: $\Theta(\quad)$

[illegible]



שאלה 2 (25 נק')

נתון מערך `arr` לא ממוין באורך n שמכיל מספרים שלמים.

ממשו את הפונקציה

```
void AlternatingArrange(int arr[], int n)
```

שתקבל את המערך `arr` ואורכו n ותשנה את המערך כך שהמספרים החיוביים והשליליים שבו יופיעו לסירוגין.

לדוגמה: אם נתון המערך

-2	5	-5	7	9	4	-7	11	12	-3	5
----	---	----	---	---	---	----	----	----	----	---

המערך הבא יהיה פלט תקין של הפונקציה

5	-2	5	-5	7	-3	9	-7	4	11	12
---	----	---	----	---	----	---	----	---	----	----

וגם המערך הבא יהיה פלט תקין (וכמו כן, קיימים עוד מערכים רבים אחרים שהם תקינים):

-7	12	-5	11	-3	9	-2	7	5	5	4
----	----	----	----	----	---	----	---	---	---	---

הערות:

- עודף מספרים חיוביים צריך להופיע בסוף המערך (כמו בדוגמאות לעיל)
- עודף מספרים שליליים, לעומת זאת, צריך להופיע בתחילת המערך.
- כל הופעה של המספר אפס במערך יכולה להיחשב כמספר חיובי או שלילי.

דרישות:

- ממשו את הפונקציה בסיבוכיות זמן $O(n)$ ובסיבוכיות מקום $O(1)$.

אם לא עמדתם בדרישות הסיבוכיות, אנא ציינו כאן את הסיבוכיות שהגעתם אליה:

זמן _____ מקום נוסף _____

```
void AlternatingArrange (int arr[], int n)
```

```
{
```

[illegible]

[illegible]

[illegible]



נתונה מחרוזת seq באורך לא ידוע, שמתחילה בסדרה של אחד או יותר מופעים של ה'X' ולאחריו מספר מופעים לא קטן יותר של ה'Y'. מספר המופעים של ה'X' לא ידוע לנו ונסמנו ב-n.

מממשו את הפונקציה `FindNumX` המקבלת את המחרוזת `seq` ומחזירה את מספר המופעים של התו `'X'` במחרוזת (כלומר את `n`).

עבור המחרוזת הבאה seq="XXXXXYYYYYY" הפונקציה תחזיר 5.

- סיבוכיות זמן $O(\log n)$ וסיבוכיות מקום $O(1)$.

אם לפי חישוביכם לא עמדתם בדרישות הסיבוכיות אנא ציינו כאן את הסיבוכיות שהגעתם אליה:

זמן _____ מקום נוסף _____

[illegible]

[illegible]

[illegible]

[illegible]



שאלה 4 (25 נקודות) :

נתונה מחרוזת `str` שמכילה אותיות אנגליות גדולות בלבד. ממשו פונקציה שמדפיסה פעם אחת כל מחרוזת שניתן להרכיב מהתווים של המחרוזת הנתונה בסדר למילוני (לקסיקוגרפי).

```
void PrintAllPossibleSubstrings(char *str)
```

דוגמה:

בהינתן מחרוזת "SAAB", הפונקציה תוציא את הפלט הבא (רק 16 מילים ראשונות מופיעות):

A

AA

AAB

AABS

AAS

AASB

AB

ABA

ABAS

ABS

ABSA

AS

ASA

ASAB

ASB

הערות:

- יש להשתמש בשיטת `backtracking` כפי שנלמדה בכיתה.
- בשאלה זו אין דרישות סיבוכיות, אולם כמקובל ב-`backtracking` יש לוודא שלא מתבצעות קריאות רקורסיביות מיותרות עם פתרונות שאינם חוקיים.
- ניתן ומומלץ להשתמש בפונקציות עזר (ויש לממש את כולן).



```
void PrintAllPossibleSubstrings(char *str)
```

```
{
```


[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]